Bauhaus-Universität Weimar

Software Engineering II **Name:** _____
Winter Semester **Matriculation number:** _____
Duration: 90 Minutes

Alle Fragen sind auf Deutsch und English angegeben.

Before you start:

- Turn off your phone.
- Have your Thoska ready.
- Write down your matriculation number on the top of the cover page and on top of every page with answers. Pages without matriculation numbers will not be graded.
- Please put away everything except your pens and Thoska.

Important notes:

- Make sure that your answers are clearly marked with the number of the corresponding question.
- Always explain your answers.
- Write your answers in full sentences.
- Provide only one solution for each question. Any additional answers will be ignored.
- Do not use your own paper. Only use the pre-printed sheets for your answers, including the back of the pages (if necessary). You can get additional sheets on request.
- Do not use a red pen or pencil. Answers written with either of those won't be graded.
- You are *not* allowed to use any supporting materials like books or personal notes.

With my signature below, I confirm that I have read and understood the notes above and that I am healthy and capable of taking the exam.

Name: _____

Matriculation number: _____

Date: _____

Signature: _____

Will be filled out by the exam supervisors

Number of additional sheets:

Question	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	Total
Points	3	6	6	6	4	4	4	4	4	6	6	4	8	4	15	4	4	92
Score																		

This exam consists of 17 questions.

Points are indicated for each question and broken down to each part.

Question 1: Software Engineering (3 points)

Definieren Sie den Begriff Software Engineering. Wählen Sie zwei Aspekte Ihrer Definition aus und veranschaulichen Sie sie jeweils an einem Beispiel.

Define the term Software Engineering. Pick two aspects of your definition and illustrate each on an example.

Marking:

1 mark for definition (0 marks if definition is incomplete), e.g.,

- "Set of methodologies, techniques, and tools to build high-quality software in a cost-effective way"
- "SE is based on a systematic approach that uses appropriate tools and techniques, operates under specific development constraints, follows a process"

1 mark each for each illustrated aspect (0 marks if software engineering is illustrated as a whole, but not the aspects), e.g.,

- tools: Example tools are IDEs and version control software.
- constraints: Software development is constrained by available people and time.

Question 2: Lifecycle Models (6 points)

Nennen Sie die traditionellen Phasen der Softwareentwicklung. Geben Sie zwei Vorgehensmodelle an und erklären Sie, wie die Phasen in dem jeweiligen Vorgehensmodellen abgebildet werden.

List the traditional software engineering phases. Provide two examples of lifecycle models and explain how all phases are reflected in these lifecycle models.

Marking:

2 marks for 5 traditional phases: Requirements engineering, Design, Implementation, Verification and validation, Maintenance (a.k.a. evolution) -0.5 for each missing/incorrect phase

2 marks for each of the lifecycle models, 1 mark if not all phases are reflected in the explanation, 0 marks if phases are ignored and only lifecycle models are mentioned

Question 3: XP vs. Waterfall Model (6 points)

Wählen Sie drei Aspekte von Xtreme Programming aus, stellen Sie diese dem Wasserfall-Modell gegenüber und erklären Sie für jeden einen von Xtreme Programming beabsichtigten Vorteil.

Pick three aspects of Xtreme Programming, contrast them with the Waterfall model, and explain the intended benefit of each aspect of Xtreme Programming.

Marking:

1 mark x 3 for mentioning and matching to corresponding aspect of waterfall model (only 0.5 if aspect is not matched to Waterfall model)

- Several rounds of development: system concept, requirements, design
- In each round, mitigate risks: Define objectives of part you are doing, Map alternatives for implementation, Recognize constraints on these alternatives, Use prototyping, analysis, etc. to gain necessary knowledge and reduce risk, Plan the next step
- At the end, perform sequence of coding, testing, and integration

1 mark for reasonable benefit

must come in triplets; (spiral, waterfall, benefit), not enough to list one benefit or general ones that do not match the contrasted elements

Question 4: Software Engineering Mistakes (6 points)

Nennen Sie 3 typische Fehler in Softwareentwicklungsprozessen aus der Vorlesung und erklären Sie für jeden, ob er nur für Softwareprodukte oder auch für einfache Programme existiert.

List 3 classic software engineering mistakes from the lecture and discuss for each whether it is specific to software products or also exists for simple programs.

Marking:

1 mark x 3 for each mistake:

- scheduling: short phases with identifying highest risks
- planning issues: see above
- gold plating requirements: many iterations and evaluation, force progress to next issue
- silver bullet: evaluation in every iteration

1 mark x 3 for the mitigation of each mistake. Mitigation has to be specific to each point (this is not a recall, but a transfer task)

Question 5: Stakeholders (4 points)

Nennen Sie vier Stakeholder für eine Software zur Steuerung eines selbstfahrenden Taxis und geben Sie für jeden ein relevantes Interesse (spezifisch für das Taxi) für die Software an.

Name four stakeholders for the software controlling a self-driving taxi and provide for each a relevant concern (specific to the taxi) for the software.

Marking:

1 mark for every stakeholder and relevant concern

if no concerns are given, at most one mark in total

- Developers need specification and have to develop software
- passengers want safe transportation
- pedestrians want safe driving behavior
- customers want high availability
- legislation want to ensure keeping traffic laws

Question 6: Requirements classification (4 points)

Bestimmen Sie für jede Anforderung, ob sie funktional oder nicht-funktional ist. Begründen Sie dies in je einem Satz.

For each of the following requirements, mark if they are functional or non-functional. Justify each answer in one sentence.

(a) *Kunden können das Taxi bestellen, um sie an einem gewünschten Ort abzuholen.*
Customers may order the taxi to pick them up at a desired place.

Marking:

functional: clearly a function to provide

(b) *Das Taxi sollte verantwortungsbewusst fahren.*
The taxi shall drive responsibly.

Marking:

non-functional: a quality of how it drives

Question 7: User stories (4 points)

Schreiben Sie zwei User Stories als Anforderungen an ein selbstfahrendes Taxi.

Write two user stories as requirements for a self-driving taxi.

Marking:

2 marks for each, where -1 mark for missing any of the three elements: role, capability, value

Example: "As a passenger, I want to be able to describe the destination so that I don't have to google the address."

Example: "As a pedestrian, I want to call the taxi by hand gestures so that I don't have to use my phone."

0 if it is not a user story

Question 8: State Chart vs. Sequence Diagram (4 points)

Nennen Sie eine Gemeinsamkeit und einen Unterschied zwischen State Charts und Sequenzdiagrammen. Geben Sie ein Beispiel an, in dem ein Diagrammtyp passender ist als der andere.

List a commonality and a difference between State Charts and Sequence Diagrams. Give an example where one diagram type would be used over the other.

Marking:

1 mark for commonality, e.g., behavior diagram, message passing

1 mark for difference, e.g., incomplete scenario vs. full behavior

2 marks for the example: 1 mark for relevance, 1 mark for correctness of choice of diagram

Question 9: Precision (4 points)

Schlagen Sie eine Anforderung mit hoher Präzision vor und geben Sie eine Variante der Anforderung mit geringerer Präzision an. Begründen Sie anhand eines knappen Szenarios, warum hier eine geringere Präzision wünschenswert sein könnte.

Suggest a requirement with high precision and provide a variant of the same requirement with lower precision. Argue why lower precision might be desired in your example requirement by providing a brief scenario.

Marking:

1 mark for the precise, 1 mark for the imprecise requirement only on in total, if the two requirements are not related

1 mark for example scenario and 1 mark for argument

Question 10: Modeling Language Elements (6 points)

Nennen und erläutern Sie (maximal einen Satz) die drei Kriterien der Modelldefinition. Veranschaulichen Sie jedes anhand eines Beispiels.

List and explain (max. one sentence) the three criteria of the definition of models. Illustrate them with an example.

Marking:

1 mark per element and 1 mark per explanation

- syntax
- semantics
- semantic mapping

only 1 mark in total if explanations are missing

Question 11: Implementation guidelines (6 points)

Nennen Sie drei Richtlinien zum Schreiben von Codekommentaren und begründen Sie jede davon knapp.

List three guidelines for writing code comments, and for each, provide a brief rationale.

Marking:

1 mark for each guideline, e.g., write opening credits, write formal assumptions, summarize complex passages, etc.

1 mark for a rationale, e.g., helps understand the purpose of the class, prevents devs from calling methods with incorrect parameters, etc.

Question 12: Testing techniques (4 points)

Nennen Sie zwei Techniken für Black-Box-Tests und beschreiben Sie jeweils einen Vorteil.

Name two black-box testing techniques and give one advantage of each.

Marking:

1 mark for each technique, e.g., metamorphic testing, exhaustive testing, combinatorial test design

1 mark for benefits each (benefits must be explicitly mentioned as such, even if part of the definition is repeated)

Question 13: Combinatorial test design (8 points)

Verwenden Sie den In-Parameter-Order-Algorithmus, um den gegebenen Testplan für eine paarweise Abdeckung ($t = 2$) zu vervollständigen.

Use the In-Parameter-Order algorithm to complete the test plan below for pairwise coverage ($t = 2$).

Parameter: $Players = \{single, multi\}$, $Controllers = \{keyboard, mouse\}$, and $Audio = \{off, music\}$.

Testplan:

$single$	$keyboard$
$single$	$mouse$
$multi$	$keyboard$
$multi$	$mouse$

Marking:

1 mark for each element added to existing rows

2 marks for completing rows (ranges from 0-2, since it is a greedy algorithm)

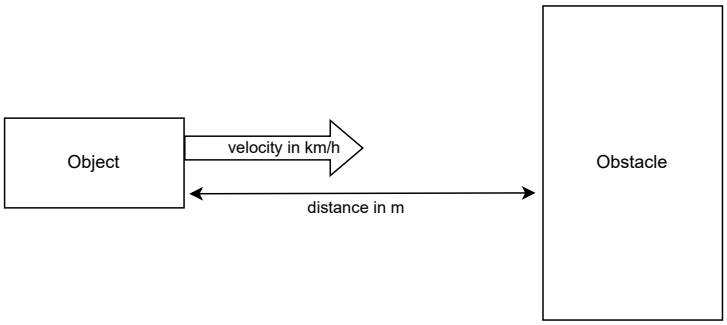
-1 for each missing pair (pair of values that is not covered)

0 if the algorithm was not followed, e.g., just listing tuples

Question 14: Metamorphic testing (4 points)

Erläutern Sie am Beispiel einer vereinfachten Kollisionsberechnung (basierend auf Geschwindigkeit und Entfernung zum Hindernis) die Anwendung metamorphen Testens. Geben Sie die metamorphische Beziehung an, welche Sie voraussetzen.

Explain how metamorphic testing may be applied to a simplified collision detection system (based on velocity and distance to an obstacle). Clearly state the metamorphic relation you assume to hold.



Marking:

This could be a probability or just a true/false output; both assumptions are fine.

2 marks the oracle, e.g., faster speed and lower distance will increase the probability of crash (or maintain the positive prediction of collision, but not the negative one)

2 marks for how to use it to compare two test outputs, e.g., feed two inputs where one changed the values (faster speed, lower distance) and compare results

Question 15: White-box Testing

(a) (6 points) *Zeichnen Sie ein Kontrollflussdiagramm für den angegebenen Code (die Zeilen sind so ausgerichtet, dass das Diagramm neben dem Code passen sollte). Die Knoten mit der Zeilennummer zu kennzeichnen ist ausreichend.*
Draw the control flow graph for the given code snippet (the lines are spaced to draw the graph next to the code). Labeling nodes with the line number is sufficient.

```
1  public boolean check(int num) {
2      int a = 5;
3      int b = num;
4      while (b != 0) {
5          int temp = b;
6          b = a % b;
7          a = temp;
8      }
9      if (a == 5) {
10         if (num == 0) {
11             return false;
12         }
13         return true;
14     }
15     return false;
16 }
```

(b) (3 points) *Geben Sie eine minimale Testsuite für eine maximale Anweisungsabdeckung an.*
Provide a minimal set of test cases for maximal statement coverage.

Marking:
Example: (0, false), (5, true), (1, false) ; (-1 if not minimal)
0 in total, if these are not test cases, i.e., if the input is not clear. It is OK, if only the input of each pair is given.
We need at least three – one for each `return`.

Marking: for each missing edge and each missing node (down to -6)

(c) (2 points) *Geben Sie eine minimale Testsuite für eine maximale Branch-Coverage an.*
Provide a minimal set of test cases for maximal branch coverage.

Marking:
Example: we can reuse the above and we cannot get better (-1 if not minimal)

(d) (2 points) *Erklären Sie die Beziehung zwischen Bedingungsüberdeckung und Entscheidungsüberdeckung.*
Explain the relation between condition coverage and branch coverage.

Marking:
incomparable: condition coverage might not cover all branches and thus not all statements (no need to define both, defining would not be sufficient for points)

(e) (2 points) *Berechnen Sie die zyklomatische Komplexität des Programms.*
Compute the cyclomatic complexity of the program.

Marking:
based on control flow graph: #Edges - #Nodes + 2*#Components
-1 if formula incorrect
-1 if computation incorrect
(computation must be based on control flow graph provided in (a))

Question 16: Agile manifesto (4 points)

Nennen Sie zwei der Grundsätze (in der Form „x über y“) des Agilen Manifests. Wählen Sie für jeden Grundsatz ein Prozessmodell aus und benennen Sie kurz, wie dieses die Werte abbildet.

List two of the values (in the form “x over y”) of the Agile manifesto. For each value, pick a software lifecycle model and briefly suggest how that model reflects the value.

Marking:

1 mark x2 per value from below (must be of the same form, but wording can be different)

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

1 mark per example contradiction, e.g., Waterfall contradicts ”responding to change” by following linear process.

Question 17: Code metrics (4 points)

Erklären Sie knapp die Metriken LOC *und* Fehler pro kLOC*. Nennen Sie jeweils ein Beispiel wofür jede Metrik im Vergleich zur anderen aussagekräftig sein könnte.*

Briefly explain the metrics *LOC* and *Bugs per kLOC*. Give one example each where the metric might be more useful than the other.

Marking:

1 mark for each explanation, LOC = lines of code, Bugs per kLOC bugs per 1000 lines of code.

1 mark for each example: LOC is an absolute measure and can indicate project sizes, while bugs per kLOC is relative. Bugs per kLOC is a better indication of quality (if measured in bugs).